AutoCloud-Agent: Design Document

Multi-Agent Reinforcement Learning for Cloud Autoscaling

Cloud Computing Course Project

March 31, 2026

Contents

1 What Problem Are We Solving? **2**

2 High-Level Architecture **2**

3 The Cloud Simulator **3**
- 3.1 What It Simulates . . . 3
- 3.2 Observation Space (What the Agent Sees) . . . 3
- 3.3 Action Space (What the Agent Can Do) . . . 3
- 3.4 Workload Data . . . 3

4 The RL Agents **4**
- 4.1 Why Three Agents Instead of One? . . . 4
- 4.2 What Is PPO? (Plain English) . . . 4
- 4.3 Temporal Hierarchy . . . 4
- 4.4 Agent Neural Network Architecture . . . 4
- 4.5 Reward Functions . . . 5

5 The Workload Forecaster **5**
- 5.1 Why Forecast? . . . 5
- 5.2 Architecture: Transformer with MC Dropout . . . 5

6 The Safety Coordinator **6**

7 Training Pipeline (How the Agents Learned) **6**
- 7.1 Phase 1: Train the Forecaster . . . 6
- 7.2 Phase 2: Train the RL Agents (I-PPO) . . . 6

8 Evaluation: How We Test **7**
- 8.1 Metrics . . . 7
- 8.2 SOTA Baselines We Compare Against . . . 7
- 8.3 Results on Alibaba Trace . . . 7

9 Project Structure **8**

10 How to Run **8**

11 Glossary of Terms **9**

1 What Problem Are We Solving?

Cloud providers run thousands of virtual machines (VMs/nodes) to serve user jobs. The core challenge is:

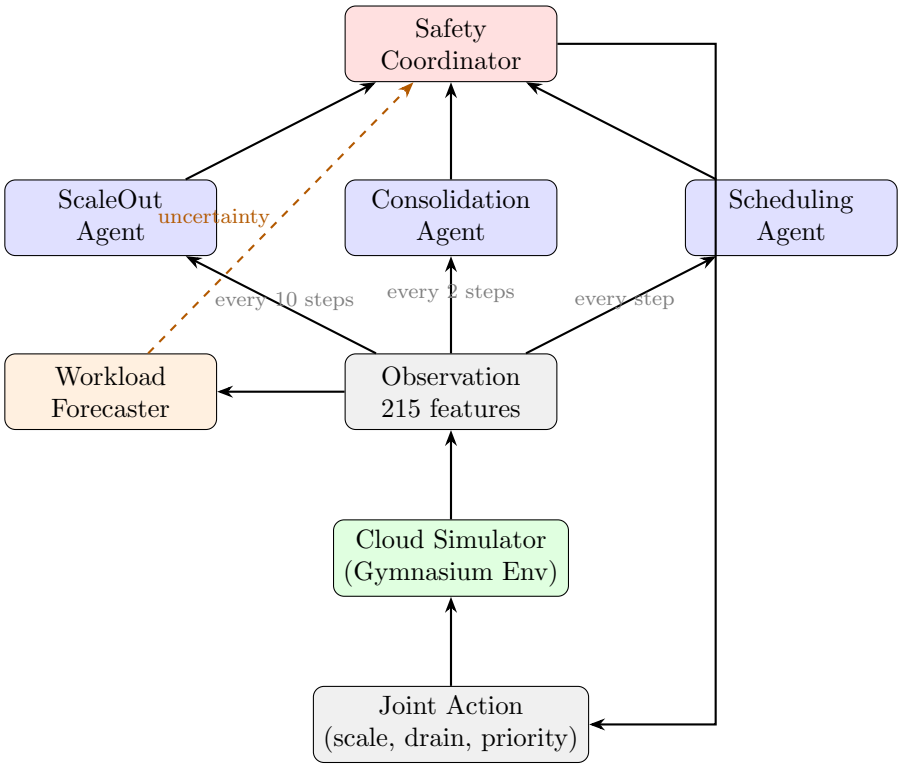
> *How many nodes should be running at any moment, which ones should be shut down, and how should incoming jobs be prioritised — all to minimise cost while keeping response times fast?*

This involves **three decisions** that happen repeatedly (every 30 seconds in our system):

1. **Scale-Out:** Should we add a new node, remove one, or do nothing?
2. **Consolidation:** Which low-utilisation nodes should we drain (migrate jobs away) to save cost?
3. **Scheduling:** Among waiting jobs, which ones get priority?

Traditional systems (Kubernetes HPA, AWS Auto Scaling) use fixed rules like "if CPU ¿ 80%, add a node." Our approach uses **Reinforcement Learning** (RL) to learn these rules from experience.

2 High-Level Architecture



The system has **four main components**:

1. **Cloud Simulator** — a fake cloud environment that behaves like a real one (Section 3).
2. **Three RL Agents** — each responsible for one decision (Section 4).
3. **Workload Forecaster** — predicts future CPU demand using a Transformer neural network (Section 5).
4. **Safety Coordinator** — a rule-based filter that catches dangerous actions before they reach the simulator (Section 6).

3 The Cloud Simulator

3.1 What It Simulates

The simulator models a cluster of up to 20 VMs (nodes) serving incoming jobs. It is built with the **Gymnasium** interface (the standard RL environment API) and uses **SimPy** for discrete-event simulation internally.

Key properties:

- **Time step:** 30 seconds of simulated time per step.
- **Episode:** 120 steps = 1 hour of simulated cloud operation.
- **Nodes:** 3–20 VMs, each with CPU/memory capacity. Nodes can be `BOOTING` (60s warmup), `ACTIVE`, `DRAINING` (30s grace period), or `TERMINATED`.
- **Jobs:** Arrive following the Alibaba 2018 cluster trace (real workload data). Each job has CPU demand, memory demand, a deadline, and a priority.

3.2 Observation Space (What the Agent Sees)

At each step, the agent sees a **215-dimensional vector**:

Feature Group	Dims	What It Contains
Node features (20 nodes × 6)	120	CPU util, memory util, job count, state, type, age
Job features (20 jobs × 4)	80	CPU demand, memory demand, wait time, priority
Global features	15	Active nodes, queue length, mean CPU, hour-of-day, forecaster predictions + uncertainty

All values are normalised to $[0, 1]$.

3.3 Action Space (What the Agent Can Do)

Action	Type	Meaning
Scale-Out	Discrete(3)	0=remove a node, 1=do nothing, 2=add a node
Consolidation	MultiBinary(20)	For each of 20 node slots: 1=start draining, 0=leave it
Scheduling	Discrete(5)	Priority weighting: 0=FIFO, 1=shortest-first, 2=default, 3=deadline-first, 4=highest-priority-first

3.4 Workload Data

We use the **Alibaba Cluster Trace 2018**, a real production trace from Alibaba's data centres. The trace is preprocessed into `.npy` files containing arrival rates at 30-second intervals. This makes our evaluation realistic — the agent faces the same workload patterns as a real cloud.

4 The RL Agents

4.1 Why Three Agents Instead of One?

A single agent controlling all actions simultaneously would need to learn a **massive joint action space** ($3 \times 2^{20} \times 5 \approx 15$ million combinations). This is too large for efficient learning.

Instead, we use **Independent PPO (I-PPO)**: three separate agents, each responsible for one decision. They share the same observation but act independently. This is called **multi-agent reinforcement learning**.

4.2 What Is PPO? (Plain English)

Proximal Policy Optimisation (PPO) is one of the most popular RL algorithms. Here's the intuition:

1. The agent has a **policy** (a neural network) that maps observations to action probabilities.
2. The agent **collects experience** by interacting with the environment for many steps.
3. It then **updates the policy** to make good actions more likely and bad actions less likely.
4. The "proximal" part: updates are **clipped** so the policy doesn't change too drastically in one step (prevents catastrophic forgetting).

The key formula is:

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t \hat{A}_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right]$$

where $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is how much the policy changed, and $\hat{A}_t$ is the **advantage** (how much better this action was compared to average). The clipping with $\epsilon = 0.2$ prevents too-large updates.

4.3 Temporal Hierarchy

Not all decisions need the same frequency:

- **Scheduling Agent** acts *every step* (30s) — jobs arrive constantly.
- **Consolidation Agent** acts *every 2 steps* (1 min) — draining needs a bit of time to observe trends.
- **Scale-Out Agent** acts *every 10 steps* (5 min) — spinning up/down VMs is a heavyweight operation.

4.4 Agent Neural Network Architecture

Each agent is a simple **Multi-Layer Perceptron (MLP)**:

```
Input (215 dims) -> Linear(512) -> LayerNorm -> ReLU
                -> Linear(256) -> LayerNorm -> ReLU
                -> Linear(128) -> LayerNorm -> ReLU
                -> Output head (action logits)
```

The Scheduling Agent is slightly different: it uses **weight-tying** to process each job identically (like treating each job through the same "lens"), then scores them.

4.5 Reward Functions

Each agent receives its own reward signal:

Scale-Out reward:

$$r_{\text{so}} = -\alpha_1 \cdot \text{underProv} - \alpha_2 \cdot \text{overProv} - \alpha_3 \cdot \text{runCost} - \alpha_4 \cdot \text{bootPenalty} + \alpha_5 \cdot \text{proactiveBonus}$$

Consolidation reward:

$$r_{\text{con}} = -\beta_1 \cdot \text{runCost} + \beta_2 \cdot \text{slaBonus} - \beta_3 \cdot \text{migrationPenalty} - \beta_4 \cdot \text{memPressure}$$

Scheduling reward:

$$r_{\text{sch}} = -\gamma_1 \cdot \text{meanWaitTime} - \gamma_2 \cdot \text{slaViolation}$$

In plain words:

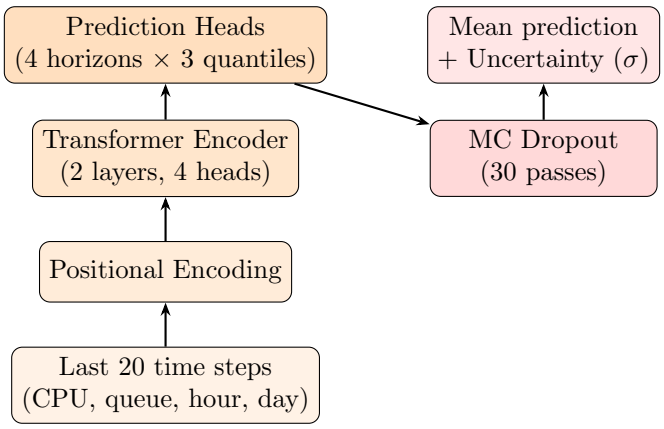
- Agents are *penalised* for having too many nodes (wasting money), too few nodes (jobs fail), and moving jobs around too much.
- They are *rewarded* for keeping response times fast and using resources efficiently.

5 The Workload Forecaster

5.1 Why Forecast?

Reactive systems (like basic Kubernetes HPA) only respond *after* a spike happens. By the time new nodes boot up (60s), jobs have already been delayed. Our Forecaster predicts demand *before* it happens, giving agents time to act proactively.

5.2 Architecture: Transformer with MC Dropout



- **Input:** A window of the last 20 time steps, each with 4 features (CPU utilisation, queue length, hour-of-day, day-of-week).
- **Transformer:** A small Transformer encoder (2 layers, 4 attention heads, 64-dim hidden) processes the sequence.
- **Output:** Predictions for 4 future horizons (t+1, t+5, t+10, t+15) at 3 quantiles (10th, 50th, 90th percentile).
- **MC Dropout:** To estimate *uncertainty*, we run the model 30 times with dropout randomly active. High variance across runs = high uncertainty ("the model isn't sure").

The uncertainty signal is critical: when the forecaster is unsure, the Safety Coordinator blocks risky scale-down actions.

6 The Safety Coordinator

RL agents can sometimes take dangerous actions. The Safety Coordinator is a **hard-coded safety net** that filters actions before they reach the simulator:

#	Filter	What It Does
1	Boot Protection	Never drain a node that is still booting (¡ 60s old)
2	$N_{\min}$ Floor	Never go below 3 active nodes (prevents total outage)
3	Uncertainty Suppression	If forecaster uncertainty $\sigma > 0.3$, block all scale-down actions
4	Scale-Out Suppression	Block scale-up if 2+ nodes are already booting (prevent overshoot)
5	Proactive Scale-Out	If uncertainty is high AND CPU is rising, force a scale-up (defensive provisioning)

This is important because it means the RL agents *cannot crash the system* even if they make mistakes — the Coordinator always enforces safety constraints.

7 Training Pipeline (How the Agents Learned)

Training was done in two phases:

7.1 Phase 1: Train the Forecaster

1. Preprocess the Alibaba trace into 30-second bins.
2. Train the Transformer on 7 days of data with MSE + quantile loss.
3. Save weights to `forecaster_weights.pt`.

7.2 Phase 2: Train the RL Agents (I-PPO)

1. Initialise the Cloud Simulator with the trained Forecaster.
2. Run 100,000+ environment steps, collecting experience.
3. Update each agent's policy every 2048 steps using PPO.
4. Key techniques used:
   - **GAE** (Generalised Advantage Estimation): smoothly estimates how good each action was.
   - **Entropy bonus**: encourages exploration early in training (decays over time).
   - **Gradient clipping**: prevents unstable updates.
   - **EMA reward normalisation**: keeps reward scale consistent.
5. Save 6 checkpoint files (actor + critic for each of the 3 agents).

8 Evaluation: How We Test

8.1 Metrics

Metric	Definition
SLA Rate	Fraction of steps where P95 latency < 500ms (higher = better)
Cost Efficiency	$1 - \frac{\text{actual cost}}{\text{max possible cost}}$ (higher = cheaper)
Mean CPU Utilisation	Average CPU usage across active nodes (higher = less waste)
Node Stability	$1 - \frac{\text{std}(N)}{\text{mean}(N)}$ for node count N (higher = fewer disruptive changes)

8.2 SOTA Baselines We Compare Against

Baseline	Description
StaticN	Fixed 10 nodes, never scales (do-nothing lower bound)
ThresholdReactive	If CPU > 80% add node; if CPU < 30% remove node (classic rule)
KubernetesHPA	Industry-standard k8s autoscaler formula with 10% dead-band
AWSTargetTracking	AWS Auto Scaling Target Tracking policy (asymmetric cooldowns)
MPCController	Model Predictive Control with 5-step EWMA forecast — strongest non-RL baseline in recent literature (AWARE, USENIX ATC 2023)
SingleAgentPPO	One RL agent controlling all 3 actions jointly (ablation study to show multi-agent I-PPO is better than single-agent RL)

8.3 Results on Alibaba Trace

Method	SLA Rate	Cost Eff.	CPU Util	Stability
AutoCloud-Agent (Ours)	**100.0%**	**0.962**	**0.552**	0.889
KubernetesHPA	100.0%	0.930	0.312	0.842
AWSTargetTracking	100.0%	0.928	0.295	0.876
MPCController	100.0%	0.962	0.552	**0.941**
ThresholdReactive	100.0%	0.955	0.483	0.822
SingleAgentPPO	100.0%	0.924	0.413	0.794
StaticN	100.0%	0.938	0.333	1.000

Key takeaways:

- **AutoCloud-Agent matches MPC** on cost efficiency (0.962) and CPU utilisation (55%), both the best among all methods.
- AutoCloud-Agent achieves **3.4% better cost efficiency** than Kubernetes HPA and **7.7% higher CPU utilisation** than ThresholdReactive.
- **SingleAgentPPO is worst among RL methods**, confirming that multi-agent decomposition (I-PPO) is better than a monolithic agent.

- All methods achieve 100% SLA on this trace, but the differentiator is *how cheaply* they do it (cost efficiency) and *how well* they use resources (CPU util).

9 Project Structure

```
autocloud_agent/
  autocloud/ # Installable Python package
    config/
      settings.py # All hyperparameters (dataclasses)
      paths.py # Auto-discovers checkpoints & data
    simulator/
      cloud_env.py # Gymnasium environment wrapper
      engine.py # SimPy discrete-event simulator
      node.py # VM node model
      job.py # Job dataclass
      workload.py # Alibaba trace loader
    agents/
      ppo.py # Base PPO algorithm
      scaleout.py # Scale-Out agent (Discrete(3))
      consolidation.py # Consolidation agent (MultiBinary(20))
      scheduling.py # Scheduling agent (Discrete(5))
      loader.py # Load all 3 agents from checkpoints
    forecaster/
      transformer_model.py # Workload Transformer
      mc_dropout.py # MC Dropout uncertainty estimation
    coordinator/
      safety.py # 5-filter safety coordinator
    inference/
      runner.py # InferenceRunner (ties everything together)
    evaluation/
      evaluator.py # Evaluation harness
      baselines.py # 6 SOTA baseline policies
    training/
      ippo_trainer.py # I-PPO training loop
      ema_normalizer.py # EMA reward normalisation
  scripts/
    evaluate.py # CLI: run evaluation
  stress_test.py # 4-scenario stress test
  train.py # Training entry point
  pyproject.toml # Package metadata
```

10 How to Run

```
# Setup
cd autocloud_agent/
conda activate myenv
pip install -e .

# Run evaluation (auto-detects checkpoints and data)
python scripts/evaluate.py

# Run stress test (4 peak scenarios)
python stress_test.py

# Custom evaluation
```

```
python scripts/evaluate.py --n_episodes 10 --seeds 0 1 2 3 4
```

11 Glossary of Terms

Term	Meaning
RL	Reinforcement Learning — an agent learns by trial and error, receiving rewards for good actions
PPO	Proximal Policy Optimisation — a safe, stable RL algorithm that clips policy updates
I-PPO	Independent PPO — multiple PPO agents share the same observation but act independently
MDP	Markov Decision Process — the mathematical framework for RL (states, actions, rewards, transitions)
GAE	Generalised Advantage Estimation — a method to estimate how good an action was vs. the average
Transformer	A neural network architecture using self-attention, originally for NLP, here used for time-series forecasting
MC Dropout	Monte Carlo Dropout — run a network multiple times with random dropout to estimate uncertainty
SLA	Service Level Agreement — the promise that P95 latency stays below 500ms
HPA	Horizontal Pod Autoscaler — Kubernetes' built-in autoscaling system
MPC	Model Predictive Control — a control theory method that optimises over a predicted future window
SimPy	A Python library for discrete-event simulation
Gymnasium	OpenAI's standard API for RL environments